LESSON 4

ESP32 SUPER STARTER KIT · 2-HOUR SESSION

Knobs and Light Sensors

Measure "how much" instead of just on or off.

Name: _____ Date: _____

TODAY YOU WILL LEARN

- The difference between **digital** and **analog**
- How to read a knob with `analogRead`
- How to draw live graphs with the **Serial Plotter**
- How to change ranges of numbers with `map()`
- How to sense light with a **photoresistor**

SESSION PLAN

0:00	Warm-up: digital vs analog
0:10	Wire up the potentiometer
0:20	Sketch 1: **L04_ReadKnob** + Serial Plotter
0:40	Sketch 2: **L04_SpeedKnob** (`map()`)
0:55	Short break
1:00	Wire up the photoresistor
1:10	Sketch 3: **L04_NightLight**
1:30	Challenges
1:55	Quiz and tidy up

PARTS TO COLLECT

- ☐ ESP32 board + USB-C cable
- ☐ 2 breadboards
- ☐ Your LEDs from Lesson 2 (red 25, yellow 26, green 27)
- ☐ 1 potentiometer (10 kΩ knob)
- ☐ 1 photoresistor (wiggly line on top)
- ☐ 1 resistor, 10 kΩ (brown-black-orange)
- ☐ 5 jumper wires

1. Digital vs analog

A button is **digital**: it's either pressed or not. Only two answers: `HIGH` or `LOW`. But lots of things in the world aren't just on or off. How loud is it? How hot? How bright? These are **analog** values: they can be anything in between.

The ESP32 has special pins that can measure **voltage**, from 0 V up to 3.3 V. `analogRead` turns the voltage into a number from **0** to **4095**.

Voltage on the pin	`analogRead` gives
0 V	0
about 1.65 V (half)	about 2048
3.3 V	4095

Digital or analog?

Sort these: a light switch, a volume knob, a thermometer, a doorbell.

_____

_____

2. Wire up the potentiometer

A **potentiometer** (or "pot") is a knob. It has three legs. The two outer legs go to 3.3 V and GND. The middle leg slides between them as you turn the knob, so its voltage goes from 0 V to 3.3 V.

Keep your LEDs from Lesson 2. You can remove the buttons if you need room.

✓	From	To
☐	ESP32 pin **3V3**	breadboard **+** rail
☐	Potentiometer, in the breadboard	each leg in its own row
☐	pot **left** leg	**+** rail (3.3 V)
☐	pot **right** leg	**–** rail (GND)
☐	pot **middle** leg	ESP32 pin **34**

> **3V3, never 5V!**
>
> The ESP32's pins can only handle **3.3 volts**. Always connect the **+** rail to **3V3**, not to the pin marked **5V** or **VIN**. Too much voltage can break the ESP32.

3. Sketch 1: Read the knob

File → New Sketch, save it as **L04_ReadKnob**, and upload. Open the **Serial Monitor** and turn the knob.

Type this in → save it as **L04_ReadKnob** — 15 lines

```
// Lesson 4: Read the knob
// Potentiometer: outer legs to 3V3 and GND, middle leg to pin 34.

const int KNOB = 34;

void setup() {
  Serial.begin(115200);
}

void loop() {
  int value = analogRead(KNOB);
  Serial.print("knob:");
  Serial.println(value);
  delay(50);
}
```

Code	What it means
`analogRead(KNOB)`	Measure the voltage on pin 34. Gives a number from 0 to 4095
`int value = …;`	Make a new variable and put the answer straight in it
`Serial.print("knob:");`	A **label** for the plotter

Now close the Serial Monitor and open **Tools → Serial Plotter**. Turn the knob slowly, then quickly. You are drawing a live graph!

> **Pin 34 is input-only**
>
> Pins 34, 35, 36 and 39 can only **read**, never write. That makes them perfect for sensors. We use them for analog inputs throughout this course.

4. Sketch 2: Speed knob

What if the knob could control the blink speed of an LED? The knob gives 0–4095, but we want a delay between 50 ms (fast) and 1000 ms (slow). The `map()` function converts one range into another.

File → New Sketch, save it as **L04_SpeedKnob**.

Type this in → save it as **L04_SpeedKnob** — 22 lines

```
// Lesson 4: The knob sets the blink speed
// Knob on pin 34, yellow LED on pin 26.

const int KNOB = 34;
const int LED = 26;

void setup() {
  Serial.begin(115200);
  pinMode(LED, OUTPUT);
}

void loop() {
  int value = analogRead(KNOB);
  int wait = map(value, 0, 4095, 50, 1000);
  Serial.print("wait:");
  Serial.println(wait);

  digitalWrite(LED, HIGH);
  delay(wait);
  digitalWrite(LED, LOW);
  delay(wait);
}
```

Code	What it means
`map(value, 0, 4095, 50, 1000)`	Turn a number between 0 and 4095 into the matching one between 50 and 1000
`int wait = …;`	Store the answer for the two `delay`s

> **Big idea: map() is like a ruler conversion**
>
> Halfway along one ruler is halfway along the other. So 2048 (half of 4095) becomes about 525 (half way from 50 to 1000). `map()` does the maths for you.

> **Try it**
>
> 1. Swap 50 and 1000 in the `map()` line. What changes?
> 2. Make the fastest blink even faster.

5. Wire up the photoresistor

A **photoresistor** changes its resistance with light: lots of light = low resistance, darkness = high resistance. The ESP32 can't measure resistance directly. So we pair it with a fixed **10 kΩ resistor** to make a **voltage divider**: a voltage that changes as the light changes.

✓	From	To
☐	one leg of the photoresistor	**+** rail (3.3 V)
☐	other leg of the photoresistor	ESP32 pin **35**
☐	10 kΩ resistor (brown-black-orange)	from the pin **35** row to the **–** rail

Bright light → higher number. Dark → lower number.

6. Sketch 3: Automatic night light

File → New Sketch, save it as **L04_NightLight**. Upload and open the **Serial Plotter**.

Type this in → save it as **L04_NightLight** — 25 lines

```
// Lesson 4: Automatic night light
// Photoresistor from 3V3 to pin 35, 10k resistor from 35 to GND.
// The green LED on 27 comes on when it gets dark.

const int LIGHT_SENSOR = 35;
const int LED = 27;
const int DARK = 1500;   // change this to suit your room

void setup() {
  Serial.begin(115200);
  pinMode(LED, OUTPUT);
}

void loop() {
  int light = analogRead(LIGHT_SENSOR);
  Serial.print("light:");
  Serial.println(light);
```

```
  if (light < DARK) {
    digitalWrite(LED, HIGH);
  } else {
    digitalWrite(LED, LOW);
  }
  delay(100);
}
```

First, find your numbers. Watch the plotter, then cover the sensor with your hand. Write down what you see:

Room light: _____ **Covered with hand:** _____

Pick a number between the two, and put it in the line `const int DARK = 1500;`. Upload again. Now the green LED should come on when you cover the sensor.

> **Big idea: calibration**
>
> Every room and every sensor is a bit different. Real engineers measure first, then choose their settings. This is called **calibration**.

7. Challenges

> **Challenge 1: Dimmer warning**
>
> Make the night light also print `"It's getting dark!"` in the Serial Monitor, but only when it is dark.

> **Challenge 2: Level meter**
>
> Use the knob to light up 0, 1, 2 or 3 LEDs, like a volume meter: green, then yellow, then red. *Hint:* `analogRead(KNOB) / 1024` *gives 0, 1, 2 or 3.*

> **Challenge 3: Secret box alarm**
>
> Put your circuit in a dark box or drawer. If somebody opens it and light gets in, flash the red LED fast and print an alarm message.

> **Bonus: Light-speed blink**
>
> Use `map()` so the **light** controls the blink speed, not the knob. Brighter = faster.

8. Quiz

Check what you learned

1. What range of numbers does `analogRead` give on the ESP32?

   ---

2. What does `map(value, 0, 4095, 0, 100)` give when `value` is 4095?

   ---

3. Why do we need the 10 kΩ resistor with the photoresistor?

Tidy up: unplug the USB cable. Keep the potentiometer and LEDs on the breadboard: you'll need them in Lesson 5.